Diaspore Build Guide

Checkpoints from October 29 to December 14, 2026

The Diaspore team

Version 0.2, October 29, 2026

Contents

1 How to use this guide **3**

2 What Diaspore is **3**
Real code, simulated world . . . 4
Where it is useful . . . 4
What it can and cannot control . . . 4

3 Scope **5**
Must-have: the course version . . . 5
Bonus, in this order . . . 5
Not now . . . 5
After the course: new failure classes . . . 6
After the course: interception mode . . . 6

4 Calendar **6**
Deadlines . . . 7
Weeks . . . 7

5 Phase 0: Ground **7**
CP0: A repo that tests itself . . . 7

6 Phase 1: The deterministic core **8**
CP1: Virtual clock . . . 8
CP2: Event queue . . . 9
CP3: One seed, many streams . . . 9
CP4: The loop . . . 10
CP5: Trace and hash . . . 10
CP6: Simulated network . . . 11

7 Phase 2: Real processes and the protocol **12**
CP7: Protocol spec v1 . . . 12
CP8: Talking to one process . . . 12
CP9: Lockstep driver . . . 13
CP10: Timers . . . 13

CP11: Storage, crash, and restart . . . 14
CP12: check-determinism . . . 14

8 Phase 3: The first real bug **15**
CP13: Config file . . . 15
CP14: Faults: delay, drop, and duplicate . . . 15
CP15: Crash schedule and partitions . . . 16
CP16: Workload and history . . . 16
CP17: Example key-value node with a planted bug . . . 17
CP18: Checker: acked writes are never lost . . . 17
CP19: .pappus, replay, and the first sweep . . . 18

9 Phase 4: Finish the must-haves **18**
CP20: Parallel dandelion and scaling numbers . . . 18
CP21: Linearizability checker with Porcupine . . . 19
CP22: One tiny raw node in another language . . . 20
CP23: Docs and release v0.1.0 . . . 20
CP24: dandelion across machines . . . 20

10 Phase 5: Bonus **21**
B2: HTML timeline viewer . . . 22
B3: GitHub Action . . . 22
B4: Go SDK helper . . . 22
B5: etcd Raft as a floret . . . 22

11 Phase 6: Wrap-up **22**

12 Glossary **23**

A Protocol v1 draft **24**
Events: Diaspore to node . . . 24
Reply: node to Diaspore . . . 24
Rules every node must follow . . . 24
Crashes and hung nodes . . . 25

B File drafts **26**
diaspore.yaml . . . 26
.pappus . . . 26

C Honest claims **27**

1 How to use this guide

This guide splits Diaspore into small checkpoints. Each one teaches one idea and ends with working, tested code.

Every checkpoint has the same shape:

- **Goal** What works at the end.
- **Why** What breaks without it.
- **Learn** The one new idea, in plain words.
- **Decide first** Questions to answer on paper before coding.
- **Tests first** Behaviors to write tests for before writing the code.
- **Done when** How you know it is time to move on.
- **Go you will meet** Packages and features you will likely use.

The rules:

1. One checkpoint at a time for each person. Do not start your next one until every "Done when" box is checked on your current one.
2. Every checkpoint has one owner, named in the team guide. Someone else reviews the pull request.
3. Write your "Decide first" answers in the pull request description before coding. A bigger decision becomes an ADR in `docs/adr/`.
4. Tests before code, every time.
5. No function signatures are given. Designing your own types is part of the learning.
6. If you read someone else's code for an idea, close it before writing yours. Then write yours from memory.

When you are stuck

1. Name the data shape in one plain sentence.
2. Write the concrete, repetitive version first, even if it looks clumsy.
3. Replace the parts that change with a variable.
4. Draw it: a memory diagram of what exists before and after one step.

2 What Diaspore is

Diaspore is a deterministic simulator for distributed systems. It is written in Go and ships as one binary.

Nodes are separate programs, written in any language. They talk to Diaspore through JSON lines on stdin and stdout. SDKs are optional helpers, not requirements.

Diaspore owns time, the network, crashes, and randomness, all driven by one seed. It sends one event to one node at a time and waits for the reply. That is what makes every run repeat exactly.

When a rule breaks, Diaspore saves a `.pappus` file. Anyone can replay it and see the same bug.

The commands are `diaspore run`, `diaspore dandelion`, `diaspore replay`, and `diaspore check-determinism`.

Honest pitch "A small, open, language-agnostic take on FoundationDB-style simulation, inspired by FoundationDB, TigerBeetle, and Maelstrom." Never "the first."

Real code, simulated world

Think of a flight simulator. The pilot is real, but the sky is simulated. Lessons from a simulated crash still hold, because the pilot makes the same decisions in a real plane.

Diaspore works the same way. The bug lives in a system's logic, and that logic can run inside Diaspore even though the world around it is fake.

Sans-I/O literally means "without input and output." The core logic never touches the network or the clock directly. It is only handed events and answers them.

Code built this way runs in Diaspore for testing and in production for real use. Only a thin layer differs: in production it talks to real sockets and clocks, and under Diaspore it talks JSON lines.

etcd's Raft library is built this way. FoundationDB and TigerBeetle test their real production code like this too.

The honest cost: a system must be built, or adapted, in this style first.

Where it is useful

- **Building replicated systems.** Leader election, replication, and consensus code hit bugs that need a crash plus bad timing. A sweep over thousands of seeds finds them.
- **Bug reports.** Instead of "it fails sometimes," a developer attaches a `.pappus` file. Anyone can replay the exact failure on their own machine.
- **Regression tests in CI.** Once a bug is fixed, its `.pappus` file becomes a test that proves it stays fixed. Bonus B3 runs a sweep on every pull request.
- **Learning distributed systems.** Students building Raft or a key-value store can see exactly why their code broke. Maelstrom already serves this group, so exact replay is the edge.

What it can and cannot control

Diaspore controls what travels between nodes, and when things happen. It cannot control what happens inside the machine: threads, disk, memory, the kernel, or containers.

Issue	v0.1, florets	v0.2, interception
A message arrives late or out of order	Yes	Yes
A message is lost, or a partition cuts nodes off	Yes	Yes
A message arrives twice	Yes	Yes
A node crashes and restarts	Yes	Partly, since disk is real
A timeout fires before a slow reply arrives	Yes	Yes
Two nodes' clocks disagree	Not yet	Possible
Two threads inside one node race each other	No	No
A disk write is lost before it is saved	No	No
Docker networking or DNS is misconfigured	No	No
A node runs out of memory or CPU	No	No
Real TCP or kernel behavior misbehaves	No	No

Here v0.1 and v0.2 are Diaspore releases, not versions of this guide.

3 Scope

Must-have: the course version

- Core: event loop, virtual time, one seed, and a trace hash.
- Protocol v1, written as a spec with JSON Schema files.
- A lockstep process driver with a watchdog.
- Timers, storage, and crash with restart.
- Faults: delay, drop, duplicate, crash, and partition.
- Workload and history, plus two checkers: acked writes and linearizability.
- A raw Go key-value example node with one planted bug.
- One tiny raw node in another language.
- CLI: `run`, `dandelion` across all cores, `replay`, and `check-determinism`.
- `dandelion` across machines: a coordinator hands out seed ranges to workers on other machines (CP24, formerly bonus B1).
- The `.pappus` v1 format.
- Scaling measurements for the report, on one machine and across machines.

Bonus, in this order

Bonus B1, `dandelion` across machines, moved into the must-haves as CP24. The other bonus items keep their numbers.

- B2: An HTML timeline viewer.
- B3: A GitHub Action.
- B4: A Go SDK helper.
- B5: etcd's Raft library, wrapped as a floret.

Not now

Python SDK, a real production runtime, disk faults, clock skew, shrinking failing runs, and binary encodings like protobuf.

After the course: new failure classes

These fit the building-block design, so you or the community can add them after v0.1.

- **Gray failure** means a floret is degraded but still looks healthy. Diaspore would create the conditions, like a slow floret, a flaky link, or a one-way partition, and the checkers would catch the damage.
- **Response failure** means a floret returns wrong values or makes an invalid state change. Diaspore would change a reply in transit, or hand back corrupted stored state on restart.
- **Byzantine failure** means a floret lies, or tells different peers different things. Diaspore would change a floret's messages differently for each recipient. Realistic lies need knowledge of the message format, so this suits community plugins.

In every case the seed decides the change, so runs stay deterministic. Tolerating Byzantine faults, with signing and special quorums, is the tested system's job, not Diaspore's.

After the course: interception mode

The long-term goal is for Diaspore to wrap unmodified programs, not only florets. Interception literally means catching a program's requests to the OS, like "what time is it" or "send this packet," and answering them yourself.

- Diaspore would answer every clock read, random number, and network call, so it dictates the program's whole environment.
- It starts Linux only, with single-threaded programs.
- Real programs ask questions in the middle of their work, so the event model needs a new kind of call: the program asks, and waits for Diaspore's answer.
- Go programs skip the C library, so `LD_PRELOAD` tricks do not work on them.
- Linux often answers clock reads without a real system call, so a simple interceptor never sees them.

This is why the loop talks to florets only through a small driver interface, decided in CP9. Interception becomes a second driver beside the process driver, and the core stays the same.

4 Calendar

The team of four starts work on Thursday, October 29, 2026. This calendar begins that day and ends on December 14.

Assumptions:

- Four people share the work. Each checkpoint is sized for 2 to 4 hours of one person. The plan assumes the team finishes four to five checkpoints a week. The hours per person are still open in the team guide.
- CP0 is done, so CP1 is next.
- Project deadlines use Vancouver time. The Vancouver clock stays at UTC-7 all year. Boston moves to EST on November 1. After that, Boston is two hours ahead, so 9am in Vancouver is 11am in Boston.
- Thanksgiving is Thursday, November 26. That week is planned light.

- The syllabus is tentative. Check Canvas and Piazza for changes.

Deadlines

Due (Vancouver)	What	Notes
Mon Nov 23, 9am	Project Liftoff: Project Proposal and Team Charter	11am Boston. Graded on on-time delivery and how actionable the plan is.
Mon Dec 14, 9am	Final Project deliverables (20%)	11am Boston
Mon Dec 14, 1–3pm	Final Project presentations	3–5pm Boston

Weeks

Week	Dates	Phase	Checkpoints	Milestone
1	Oct 29–Nov 1	Core	CP1, CP2, CP3, CP7, in parallel (4 days)	
2	Nov 2–8	Core, then protocol	CP4–CP6, CP8, CP9	A: Nov 8
3	Nov 9–15	Protocol	CP10–CP14	B: Nov 15, Gate 1
4	Nov 16–22	First bug	CP15–CP17, CP22, and Project Liftoff	Liftoff due Nov 23
5	Nov 23–29	First bug (light week)	CP18, CP19, CP21, and the CP24 design	C: Nov 29
6	Nov 30–Dec 6	Finish and scale out	CP20, build CP24, then CP23	D: Dec 6, Gate 2
7	Dec 7–13	Wrap-up	Report, poster, demo, buffer	Due Dec 14

Project Liftoff Draft the Project Proposal and Team Charter by Wednesday, November 18, and submit by Sunday, November 22. This guide, the team guide, and `docs/plan.md` are the source material.

Gate 1, Sunday, November 15 If Milestone B is not working, drop bonus items B2 to B5 now. Keep going on must-haves only. CP24 stays, and it has a smaller fallback (Gate 2).

Gate 2, Sunday, December 6 If Milestone D is not done, shrink CP24 but keep its core: the coordinator re-sends a dead worker's seeds to another worker. Drop workers joining in the middle of a sweep, and tuning the range sizes. Dead-worker recovery is the last thing to cut. If it is cut, write it up as a limit. Every other must-have ships in full.

5 Phase 0: Ground

CP0: A repo that tests itself

Planned: Done

Goal An empty Go project where every push runs the tests.

Why Without it, you find broken code days later.

Learn CI, short for continuous integration, literally means a server that runs your checks on every push.

Decide first

? What are the module path and the repo name?

? Apache-2.0 or MIT? Apache-2.0 adds a patent grant. Both are common for developer tools.

? Read the repository layout in `docs/design.md`. Can you say what each folder is for, in one sentence each?

Tests first

☐ One trivial test that passes.
☐ One test you break on purpose, to watch CI fail.

Done when

☐ CI goes red on a broken test and green after the fix.
☐ CI runs on Linux, macOS, and Windows.
☐ The README has the one-line pitch.

Go you will meet `go mod`, `go test`, `go vet`, and a GitHub Actions workflow file.

6 Phase 1: The deterministic core

Everything in this phase runs inside one Go process. There is no JSON and no child process yet.

One rule holds for the whole phase: no goroutines and no real clock in the core.

CP1: Virtual clock

Planned: October 29 to November 1

Goal A clock that moves only when the simulator moves it.

Why Real time is different on every run, so replays could never match.

Learn Virtual time literally means time is just a number the simulator owns. Ten simulated minutes can pass in one real second.

Decide first

? Say the clock's data shape in one sentence.
? Milliseconds or microseconds? A plain integer or a duration type? Write down why.
? Who may move the clock? What happens if someone tries to move it backward?

Tests first

☐ The clock starts at zero.
☐ Moving forward works.
☐ Moving backward is refused.

Done when

☐ All tests pass.
☐ Nothing in the package reads the real clock.

Go you will meet `time.Duration`, `int64`, and the `errors` package.

CP2: Event queue

Planned: October 29 to November 1

Goal A queue that always hands back the earliest event.

Why Events are created out of order, but they must happen in time order.

Learn A priority queue literally means a line where the most urgent item always comes out first. Here, most urgent means earliest.

Decide first

- **?** What does every event need to carry? Say it in one sentence.
- **?** Two events at the same time: which goes first? Why must that answer never depend on luck?
- **?** Values or pointers in the queue? Draw both in memory before choosing.

Tests first

- ☐ Events come out in time order.
- ☐ Same-time events come out in the order they were added.
- ☐ Taking from an empty queue behaves the way you decided.
- ☐ 10,000 inserts made from a fixed seed come out sorted.

Done when

- ☐ All tests pass.
- ☐ You can draw the heap after five inserts, by hand.

Go you will meet `container/heap`.

CP3: One seed, many streams

Planned: October 29 to November 1

Goal Every random choice in a run comes from one seed.

Why One stray random call anywhere breaks replay.

Learn A PRNG, or pseudo-random number generator, literally means a formula that makes numbers look random but repeats exactly from the same seed.

Decide first

- **?** One generator for everything, or one child stream per purpose, like network, faults, workload, and each node?
- **?** If you add a new fault type later, should every old seed's network delays change? Let your answer pick the design.
- **?** How do you turn the master seed plus a name like "network" into a child seed, the same way on every machine?

Tests first

- ☐ The same seed gives the same first 1,000 numbers.

☐ Different seeds give different numbers.
☐ Adding a new child stream does not change the existing ones.

Done when

☐ All tests pass.
☐ No global random functions are used anywhere. Check with grep.

Go you will meet `math/rand/v2`, whose PCG generator takes explicit seeds, and `hash/fnv`.

CP4: The loop

Planned: Week of November 2

Goal A loop that takes the next event, runs its handler, and repeats until the run ends.

Why This is the heart of Diaspore. Everything else plugs into it.

Learn A discrete-event simulation literally means jumping from one event straight to the next, skipping the empty time between them.

Decide first

? What does a handler receive, and what may it do? Can it call another node directly, or only schedule events?
? When does a run stop? List every reason.
? Where do test nodes live for now?

Tests first

☐ Two fake nodes play ping-pong ten times.
☐ The final virtual time equals the sum of the delays you set.
☐ Scheduling an event in the past is refused.
☐ The run stops at its time limit.

Done when

☐ All tests pass.
☐ The loop has no goroutines.

CP5: Trace and hash

Planned: Week of November 2

Goal Every run writes a trace, and the trace gets a fingerprint.

Why You need proof that two runs were exactly the same.

Learn A hash literally means a short fingerprint of some data. Change one byte and the fingerprint changes.

Decide first

? What goes in one trace line? Enough to debug, and nothing more.

? Will the same event always turn into the same bytes? Think about map key order and number formatting.

? Hash while writing, or after the run?

Tests first

☐ The same seed twice gives the same hash.

☐ Changing one delay changes the hash.

☐ Every trace line parses back.

Done when

☐ The ping-pong test also checks the hash.

Go you will meet `encoding/json`, `crypto/sha256`, `io.MultiWriter`, and `bufio`.

CP6: Simulated network

Planned: Week of November 2

Goal Messages between nodes travel through Diaspore with a seeded delay.

Why The network causes most distributed bugs, so Diaspore must control it.

Decide first

? What does the network hold? Say it in one sentence.

? Where does the delay range come from for now?

? Random delays already let messages overtake each other. Do you still need a separate reorder fault? Write your answer in the pull request.

Tests first

☐ Every message arrives inside its delay range.

☐ The same seed gives the same arrival times.

☐ With a wide delay range, some messages overtake others.

Done when

☐ Ping-pong runs over the simulated network.

Milestone A — Sunday, November 8: Deterministic core

☐ Ping-pong runs over the simulated network.

☐ The same seed gives the same hash, every time.

☐ The core has no goroutines and no real clock.

☐ `PROTOCOL.md` is written.

☐ Optional: draft blog post 1, on why distributed bugs are hard to replay.

7 Phase 2: Real processes and the protocol

Now nodes become real programs. The protocol is the product, so it gets written down first.

CP7: Protocol spec v1

Planned: October 29 to November 1

Goal A written spec that someone could implement without asking you anything.

Why SDKs are optional, so the spec is what people build against.

Learn JSON Schema literally means a JSON file that describes what valid JSON looks like.

Decide first

- **?** Start from the draft in Appendix A. What would you change?
- **?** How does a node learn the protocol version? What happens on a mismatch?
- **?** Which reply fields are required? What should a node do with an event type it does not know?

Tests first

- ☐ The schema accepts every example in Appendix A.
- ☐ The schema rejects five broken examples that you write.

Done when

- ☐ `PROTOCOL.md` and the schema files are in the repo.

Go you will meet A JSON Schema validator library for your tests, if you want one.

CP8: Talking to one process

Planned: Week of November 2

Goal Start a node program, send it `init`, and read its reply.

Why Nodes are separate programs, so Diaspore drives them through pipes.

Learn A pipe literally means a one-way stream of bytes between two programs. A program reads from its stdin and writes to its stdout.

Decide first

- **?** Who owns the process and its pipes? What gets closed, and when?
- **?** How do you read exactly one JSON line? What if a line is very long?
- **?** Where do node logs go, so stdout stays clean for the protocol?

Tests first

- ☐ A tiny raw test node, written in Go with no SDK, answers `init`.
- ☐ A node that prints garbage gives a clear error.
- ☐ A node that exits early gives a clear error.

Done when

☐ All tests pass on all three operating systems in CI.

Go you will meet `os/exec`, `bufio.Scanner` (note its default line limit of 64 KB), `bufio.Writer` with `Flush`, and `encoding/json`.

CP9: Lockstep driver

Planned: Week of November 2

Goal Send one event, wait for its reply, apply it, and repeat.

Why Without lockstep, processes race each other and runs stop matching.

Learn Lockstep literally means moving one step at a time, together, never ahead. A seam literally means a boundary where one piece can be swapped without touching the rest. In Go, a seam is usually an interface.

Decide first

- **?** What happens to the `send` list in a reply? Say it in one sentence.
- **?** A reply has sends, timers, and storage. Does the order you apply them in matter?
- **?** How long does the watchdog wait in real time? What does a timeout mean for the run?
- **?** What is the smallest interface between the loop and a floret driver? Keep the process driver behind it, so a second driver, like interception, can be added later without changing the loop.

Tests first

- ☐ Three test nodes pass a message around a ring.
- ☐ Twenty runs of one seed give one hash.
- ☐ A node that never replies trips the watchdog with a clear message.

Done when

- ☐ All tests pass.
- ☐ The watchdog can stop a run, but it never changes the run's results.

Go you will meet Goroutines and `select`, but only in the process driver, never in the loop. Also `context.WithTimeout`.

CP10: Timers

Planned: Week of November 9

Goal Nodes can set and cancel timers.

Why Heartbeats, retries, and election timeouts all need timers.

Decide first

- **?** Setting a timer whose id already exists: replace it, or refuse?
- **?** Canceling a timer that already fired: ignore it, or refuse?
- **?** A timer and a message at the same instant: which goes first? Your CP2 tie-break should answer this.

Tests first

- ☐ A timer fires exactly at the current time plus its delay.
- ☐ Canceling stops it from firing.
- ☐ Setting it again moves it.
- ☐ A node with a 100 ms heartbeat, run for one second, sends the number of heartbeats you predicted on paper.

Done when

- ☐ All tests pass.

CP11: Storage, crash, and restart

Planned: Week of November 9

Goal Nodes can save state that survives a crash.

Why Many real bugs appear only after a crash and a restart.

Learn Durable literally means it survives a crash. Memory does not.

Decide first

- **?** What does Diaspore keep for each node's storage? One sentence.
- **?** Does `store` merge keys, or replace everything?
- **?** On a crash, what happens to messages already on their way to that node? To its timers?
- **?** On restart, start a fresh process or reuse the old one? Which is more honest?

Tests first

- ☐ A counter node crashes, restarts, and continues from its stored value.
- ☐ Messages and timers behave the way you decided.
- ☐ Crash and restart show up in the trace.

Done when

- ☐ All tests pass.

CP12: check-determinism

Planned: Week of November 9

Goal A command that runs one seed twice and compares the results.

Why In any language, a node can break determinism by accident. This catches it.

Decide first

- **?** Report only "different," or point to the first line that differs? Which helps more?
- **?** Which exit code means "not deterministic"?
- **?** The `flag` package or a CLI library? Write down why.

Tests first

☐ A well-behaved node passes.
☐ A node that reads the real clock fails, and the report points at the first differing line.

Done when

☐ `diaspore check-determinism --seed 42` works on the ring test.

Milestone B — Sunday, November 15: Real processes in lockstep (Gate 1)

☐ Real node processes run in lockstep, with timers.
☐ Crash and restart work, with storage.
☐ `check-determinism` catches a bad node.
☐ Publish blog post 1.

If this is not working, drop bonus items B2 to B5 now. CP24 stays, with its smaller fallback.

8 Phase 3: The first real bug

By the end of this phase, a sweep finds the planted bug and a `.pappus` file replays it every time.

CP13: Config file

Planned: Week of November 9

Goal `diaspore.yaml` describes a whole run.

Why Runs must be shareable and repeatable without long command lines.

Decide first

? Which fields are required, and what are the defaults?
? How do you report a bad field so the user can fix it fast?
? Which YAML library? Check that it is still maintained before picking.

Tests first

☐ The example in Appendix B loads.
☐ Unknown fields are rejected.
☐ A bad duration like "5 seconds" is rejected with a clear message.

Done when

☐ All tests pass.
☐ `diaspore run` reads everything it needs from the file.

CP14: Faults: delay, drop, and duplicate

Planned: Week of November 9

Goal Rules from the config change what the network does.

Why Bugs hide behind unlucky delivery, so Diaspore must create that bad luck on purpose.

Learn A fault rule literally means three things: which messages, what goes wrong, and how often.

Decide first

- **?** Say a rule's shape in one sentence.
- **?** If two rules match one message, do both apply? In what fixed order?
- **?** Which random stream do faults draw from?

Tests first

- ☐ Drop with chance 1 drops every matching message. Chance 0 drops none.
- ☐ Duplicate delivers a message twice.
- ☐ Rules touch only the messages they match.
- ☐ The same seed gives the same drops.

Done when

- ☐ Every fault shows up in the trace, along with the rule that caused it.

CP15: Crash schedule and partitions

Planned: Week of November 16

Goal Crash nodes on a seeded schedule, and split the network into groups.

Why Crashes and partitions are where replication bugs live.

Learn A network partition literally means some nodes cannot reach others for a while.

Decide first

- **?** Say a partition's shape in one sentence.
- **?** Messages that cross a partition: dropped, or held until it heals?
- **?** How are crash and partition times picked from the seed?

Tests first

- ☐ During a partition, no message crosses between groups.
- ☐ After it heals, messages flow again.
- ☐ The same seed gives the same schedule.

Done when

- ☐ All tests pass.

CP16: Workload and history

Planned: Week of November 16

Goal Virtual clients send key-value requests and record what happened.

Why Checkers can only judge what was recorded.

Learn A history literally means the list of what every client asked, what it got back, and when.

Decide first

- **?** What is one history entry? One sentence.
- **?** When does a client give up waiting, and what do you record then? A timed-out write may or may not have happened.
- **?** How do clients pick keys and values from the seed?

Tests first

- ☐ There is one history entry per request.
- ☐ Timeouts are recorded as unknown, not as failed.
- ☐ The same seed gives the same history.

Done when

- ☐ `history.jsonl` is written for every run.

CP17: Example key-value node with a planted bug

Planned: Week of November 16

Goal A small replicated key-value node in Go, speaking raw JSON with no SDK.

Why Diaspore needs a real target, and a known bug proves the tool works.

Decide first

- **?** Describe the replication design in three sentences.
- **?** Pick the planted bug. Choose one that only shows up under a crash plus unlucky timing.
- **?** How does each node know which one is the primary?

Tests first

- ☐ With no faults, every write can be read back.
- ☐ The node passes `check-determinism`.

Done when

- ☐ Fault-free runs are clean.

CP18: Checker: acked writes are never lost

Planned: Week of November 23 (light week)

Goal A checker that catches the planted bug.

Why Without a rule to check, a broken run looks like any other run.

Learn An invariant literally means a rule that must always stay true.

Decide first

- **?** Write the rule in one exact sentence.

? How do unknown outcomes count?

? Check once at the end of the run, or after every read?

Tests first Use hand-written histories:

☐ A clean history passes.

☐ A history with a lost acked write fails.

☐ A history with an unknown write does not fail by mistake.

Done when

☐ Clean runs pass, and the planted bug fails.

CP19: .pappus, replay, and the first sweep

Planned: Week of November 23 (light week)

Goal Sweep seeds until the bug appears, save it, and replay it.

Why This is the whole promise of Diaspore, working end to end.

Decide first

? What must a `.pappus` file hold to stand alone? Start from Appendix B.

? If the node program changed since the file was made: warn, or refuse?

? Should a sweep stop at the first failure, or collect all of them?

Tests first

☐ Writing and then reading a `.pappus` file gives back the same content.

☐ Replay gives the same trace hash as the original run.

☐ Replay warns when the node program changed.

Done when

☐ `diaspore dandelion --seeds 1-500` finds the bug.

☐ `diaspore replay` shows it every time.

Milestone C — Sunday, November 29: The first bug, end to end

☐ A sweep finds the planted bug.

☐ The `.pappus` file replays it with the same hash.

☐ Record a short demo: sweep, failure, replay.

☐ Optional: blog post 2, on finding the first bug.

9 Phase 4: Finish the must-haves

CP20: Parallel dandelion and scaling numbers

Planned: Week of November 30

Goal Run many seeds at once across CPU cores, and measure how it scales.

Why This is the scalability half of the course story.

Learn Scaling up literally means using more of one machine. Scaling out means using more machines, which is CP24.

Decide first

- **?** Parallelism lives between seeds, never inside one run. Why?
- **?** How many workers? How are results collected? How do all workers stop at the first failure?
- **?** What will you measure? At least: seeds per second against worker count, events per second per run, and time spent waiting on node processes.

Tests first

- ☐ A parallel sweep finds the same failing seeds as a serial sweep.
- ☐ Results do not depend on the number of workers.

Done when

- ☐ You have a chart of seeds per second for 1, 2, 4, and 8 workers.

Go you will meet Goroutines, channels, `sync.WaitGroup`, context cancellation, and `runtime.NumCPU`.

CP21: Linearizability checker with Porcupine

Planned: Week of November 23 (light week)

Goal A stronger checker for the key-value store.

Why "No lost writes" misses bugs like stale reads.

Learn Linearizability literally means every operation looks like it happened at a single instant between its start and its end, as if there were only one copy of the data.

Decide first

- **?** Read Porcupine's README first. How will you describe the key-value store as a model?
- **?** How do unknown outcomes map onto that model?
- **?** Check all keys together, or one key at a time?

Tests first

- ☐ A hand-made valid history passes.
- ☐ A history with a stale read fails.

Done when

- ☐ The checker can be turned on by name in `diaspore.yaml`.

Go you will meet `github.com/anishathalye/porcupine`.

CP22: One tiny raw node in another language

Planned: Week of November 16

Goal Prove "any language" with a small node written outside Go.

Why Without it, the honest wording is only "designed for any language."

Decide first

- **?** JavaScript or Python?
- **?** What are that language's determinism traps? Python randomizes string hashing per process. JavaScript's `Math.random` cannot be seeded.
- **?** How will Diaspore set up each node's environment to avoid those traps?

Tests first

- ☐ The node passes `check-determinism`.
- ☐ It runs in one cluster alongside Go nodes.

Done when

- ☐ A CI job runs the mixed-language cluster.

CP23: Docs and release v0.1.0

Planned: Week of November 30

Goal A stranger can install Diaspore and watch a bug replay within ten minutes.

Why Open-source tools live or die by their first ten minutes.

Decide first

- **?** How do people install it: `go install`, release binaries, or both?
- **?** What goes in the README, and in what order?
- **?** Which honest comparison lines go in? Use Appendix C.

Done when

- ☐ A classmate follows the README with no help from you.
- ☐ v0.1.0 is tagged, with binaries for Linux, macOS, and Windows.

Go you will meet GoReleaser, if you choose release binaries.

CP24: dandelion across machines

Planned: Design in the week of November 23. Build in the week of November 30.

Goal A coordinator hands out seed ranges to workers on other machines, collects the failures, and survives a dead worker.

Why One machine only shows scaling up. This is the scale-out half of the course story, and it is the part of Diaspore that is itself a distributed system. A few workers cannot prove internet scale. They can show a trend and a bottleneck, so the report claims only that.

Learn A coordinator literally means the one process that hands out the work and collects the results. A worker does the work. Re-running a seed is safe, because the same seed always gives the same run.

Start early It only needs single runs and the `.pappus` file, so write the design in week 5. Answer the questions below, and draft the two messages: coordinator to worker, and worker to coordinator. Put them in a pull request.

Decide first

- **?** How do workers talk to the coordinator?
- **?** What happens when a worker dies in the middle of a range?
- **?** How do you make sure no seed is skipped, and none is counted twice?
- **?** How does a worker check that it has the same Diaspore version and floret programs as the coordinator?
- **?** Where do the workers run: laptops, cloud instances, or containers? Only separate machines count as scaling out. Say which one you used in the report.
- **?** What will you measure? At least seeds per second against worker count across machines, and the cost of handing out work.
- **?** Where does it stop scaling? Find the bottleneck: the coordinator handing out work, the network, or the lockstep round trip to each floret.

Tests first

- ☐ A sweep across workers finds the same failing seeds as a single-machine sweep.
- ☐ When a worker is stopped in the middle of a range, its seeds go to another worker.
- ☐ Each failing seed is reported once, with its `.pappus` file.

Done when

- ☐ The sweep runs on at least two machines, with at least four workers in total.
- ☐ You have a chart of seeds per second for one machine and for several machines.
- ☐ You can name the bottleneck, with evidence from the chart.

Go you will meet `net/http` or `net/rpc`, `context`, and timeouts.

Milestone D — Sunday, December 6: Every must-have is done (Gate 2)

- ☐ Every must-have works and is tested.
- ☐ `dandelion` runs across at least two machines and four workers, with scaling charts for one machine and for several, and the bottleneck named.
- ☐ v0.1.0 is tagged.

If this is not done, shrink CP24 as Gate 2 says.

10 Phase 5: Bonus

Planned: No scheduled week. Only if Milestone D lands early, or in the buffer days.

B1 moved into the must-haves as CP24, so the bonus items start at B2. Do these in order. They never take time from a must-have.

B2: HTML timeline viewer

One static HTML file that reads `trace.jsonl` and draws which node sent what, and when.

B3: GitHub Action

Runs a sweep on every pull request, fails the build when a rule breaks, and uploads the `.pappus` file.

B4: Go SDK helper

Only once the protocol has stopped changing.

B5: etcd Raft as a floret

A thin floret around etcd's Raft library. The library has no clock or network of its own, so its real logic runs under Diaspore unchanged. This is the first real, widely used code inside Diaspore.

- **?** How do `init`, `message`, and `timer` events turn into Raft's `Tick` and `Step` calls?
- **?** What must go into `store` so a restarted node recovers correctly?
- **?** The library is well tested. If a sweep fails, is the bug in Raft or in your wrapper? How do you tell?

11 Phase 6: Wrap-up

Planned: December 7 to 13. Deliverables are due Monday, December 14, 9am Vancouver, which is 11am Boston.

Report outline:

1. The problem: why distributed bugs do not replay.
2. The design: virtual time, one seed, lockstep, and the protocol.
3. Why Diaspore is a distributed system (see below).
4. Faults and checkers.
5. Results: the bug found, the scaling charts for one machine and for several, and the cost of the process round trip.
6. Limits, stated honestly, plus future work on gray, response, and Byzantine failures, and on interception mode.
7. Related work: FoundationDB, TigerBeetle's VOPR, Maelstrom, gosim, detsim, and Antithesis.

Why Diaspore is a distributed system The report needs one section that answers this question. Make these points, with evidence from CP24:

- There are several machines and no shared memory. The coordinator and the workers talk only by messages.
- Part of the system can fail while the rest keeps going. A worker can die in the middle of a range. The coordinator must notice, and re-send the seeds.

- Recovery is simple because of determinism. Re-running a seed gives the same run. MapReduce relies on the same idea when it re-runs a task.
- Diaspore orders events without a real clock. It uses virtual time and a fixed tie-break. This is the question Lamport's time paper asks.
- Adding workers raises throughput until something becomes the bottleneck. The charts show where.
- The checkers apply course ideas to the tested systems: replication, and linearizability.

Say what the charts show: a trend and a bottleneck. Do not claim internet scale.

Demo script: sweep, failure, `.pappus` file, replay, and the timeline if B2 exists.

Also publish blog post 3.

Final project deliverables, due Monday, December 14, 9am Vancouver

- A link to the shared repository. The README says what Diaspore is and why you built it. The commits, issues, and pull requests show how the project moved over time.
- A project management write-up: how the design moved from the first plan to the final state, who worked on what, and the problems on the way.
- An experiments write-up as a PDF, at most five pages. For each experiment: the purpose, the tradeoff, and the limits. Then the results with charts, and the analysis. The experiment code is in the repository.
- A solo reflection from each person, at most five pages, on what that person learned. It is not a team effort. Keep your own work visible in commits, pull requests, and issues.
- The syllabus also lists a poster, a blog report, and the presentation. Details come later in the semester.

Final project presentations Monday, December 14, 1 to 3pm Vancouver, which is 3 to 5pm Boston.

Wrap-up plan Freeze the code and the experiment runs by Wednesday, December 9. Write after the freeze. Keep Saturday and Sunday as buffer, and submit by Sunday night.

12 Glossary

Checker A program that reads a history and says whether a rule was broken.

CI Continuous integration: a server that runs your checks on every push.

Determinism The same inputs always give the same outputs.

Discrete-event simulation Jumping from one event to the next, skipping the empty time between.

Durable Survives a crash.

Hash A short fingerprint of some data.

History What every client asked, what it got back, and when.

Interception Catching a program's requests to the OS and answering them yourself.

Invariant A rule that must always stay true.

Linearizability Every operation looks like it happened at one instant between its start and its end.

Lockstep One step at a time, together, never ahead.

Network partition Some nodes cannot reach others for a while.

.pappus Diaspore's replay file: the seed, the setup, and the failure in one place.

Pipe A one-way stream of bytes between two programs.

Raft A set of rules for a group of servers to agree on one ordered list of changes, even when some crash.

PRNG A formula that makes numbers look random but repeats exactly from the same seed.

Sans-I/O Core logic that never touches the network or clock directly, and is only handed events.

Seam A boundary where one piece can be swapped without touching the rest.

Seed The one number that decides every random choice in a run.

Trace The list of every event in a run.

Virtual time Time as a number the simulator owns.

Watchdog A real-time timer that stops a run if a node hangs.

A Protocol v1 draft

This is the starting point for CP7, not the final spec. Each line is one JSON object, and every event gets exactly one reply.

Events: Diaspore to node

Start or restart:

```
{"type": "init", "node": "n1", "peers": ["n2", "n3"], "rand_seed": 918273, "time": 0, "
    stored": {}}
```

A message arrives:

```
{"type": "message", "time": 1250, "from": "n2", "body": {"op": "put", "key": "x", "
    value": 5}}
```

A timer fires:

```
{"type": "timer", "time": 1750, "id": "election"}
```

Reply: node to Diaspore

```
{"type": "done", "send": [{"to": "n2", "body": {"op": "ack"}}], "set_timers": [{"id": "
    election", "after": 500}], "cancel_timers": ["heartbeat"], "store": {"term": "3"}, "
    log": ["became leader"]}
```

Rules every node must follow

- Do work only inside the handler for the current event.
- Never read the real clock. Use the `time` field.
- Never open your own network connections. Use `send`.

- Never use unseeded randomness. Use `rand_seed`.
- Keep anything that must survive a crash in `store`.

Crashes and hung nodes

A crash kills the node process, so its memory is gone. A restart starts a new process and sends `init` with `stored` filled in.

If a node does not reply, a real-time watchdog stops the run and reports it. The watchdog only detects hangs. It never affects the simulation.

B File drafts

diaspore.yaml

```
version: 1
nodes:
  count: 3
  command: ./kvnode
workload:
  type: kv
  clients: 3
  ops: 200
faults:
  - fault: delay
    amount: 1ms-50ms
  - fault: drop
    chance: 0.02
  - fault: crash
    every: 5s-20s
    restart_after: 1s-3s
checkers:
  - acked-writes
run:
  duration: 120s
```

.pappus

```
{
  "pappus": 1,
  "diaspore": "0.1.0",
  "seed": 48213,
  "config": {},
  "nodes": {"kv": {"command": "./kvnode", "sha256": "..."}},
  "trace_hash": "sha256:...",
  "failure": {"checker": "acked-writes", "time": 83120, "reason": "write x=5 was acked,
     then lost after n1 crashed"}
}
```

The `config` field holds the full resolved setup, so the file stands alone. The node hash warns you if the code changed since the bug was found.

C Honest claims

Do not say	Say instead
"The first DST tool for any language"	"Inspired by FoundationDB, TigerBeetle, and Maelstrom"
"Fully deterministic"	"Deterministic when nodes follow the protocol rules, checked by trace hashes"
"Supports any language" (before CP22)	"Designed for any language"
"Test any service"	"Test nodes written against the Diaspore protocol"
"Finds real production bugs"	"Found the bug planted in the example," until it truly finds others
"Works with real systems"	"Runs etcd's Raft library through a thin floret," only once B5 works
"Wraps any program"	"Wrapping unmodified programs is future work"
"Tests real systems"	"Tests real logic written in sans-I/O style"

Before publishing, check two things: whether Maelstrom supports exact replay from a seed, and the current features of gosim and detsim.